

데이터 전처리

결측치 / 이상치 / 중복데이터

결측치(Missing Value)

- 결측치란?
 - 데이터셋에서 값이 누락된 상태
 - 설문조사에서 응답자가 특정 질문에 답하지 않거나, 시스템 오류로 데이터가 누락되는 등 다양한 이유로 발생할 수 있음
 - 결측치가 있는 데이터를 그대로 분석에 사용하면 왜곡된 결과를 초래할 수 있음

결측치(Missing Value)

- 결측치 찾아내는 방법

- NumPy 데이터 유형의 경우, np.nan이 누락된 데이터를 나타냄
- df.isnull() / df.isna() : 결측치 위치에 True를 반환하는 DataFrame을 생성
- df.isnull().sum() : 열(Column)별 결측치 개수 # 가장 일반적인 방법

```
# 각 열의 결측치 개수 확인  
df.isnull().sum()
```

결측치 처리 전략 1: 제거 (dropna)

- `df.dropna()`: 결측치가 하나라도 있는 행을 모두 제거.
 - `axis=0` 옵션: 결측치가 있는 행을 제거.
 - 언제 사용하는가?
 - : 데이터가 충분히 많고,
 - : 결측치가 있는 행의 비율이 낮을 때.
 - : 또는 특정 열이 거의 다 비어있을 때.

```
data.dropna(how = 'any', axis = 0, inplace = True)  
data
```

결측치 처리 전략 2: 채우기 (fillna)

- `df.fillna(0)`: 특정 값으로 채우기
 - `df['Age'].fillna(df['Age'].mean())`:
 - : 다른 값(평균, 중앙값, 최빈값 등)으로 채우기. → 가장 일반적인 결측치 처리 방법
 - 어떤 값으로 채울까?
 - : 숫자형 데이터: 평균(mean) 또는 중앙값(median).
 - : 범주형 데이터: 최빈값(mode).

결측치 처리 전략 2: 채우기 (fillna)

- `df.fillna(0)`: 특정 값으로 채우기

```
# age 열의 결측치를 age 열의 평균값으로 채우기
mean_age = df['age'].mean()
df['age'].fillna(mean_age, inplace=True)
```

```
print(f"age 열의 평균값: {mean_age:.2f}")
```

```
# embarked 열의 최빈값 찾기
mode_embarked = df['embarked'].mode()[0]
df['embarked'].fillna(mode_embarked, inplace=True)
```

```
# embark_town 열의 최빈값 찾기
mode_embark_town = df['embark_town'].mode()[0]
df['embark_town'].fillna(mode_embark_town, inplace=True)
```

```
print(f"embarked 열의 최빈값: {mode_embarked}")
print(f"embark_town 열의 최빈값: {mode_embark_town}")
```

불필요한 데이터 제거 (drop)

- `df.drop('column_name', axis=1)`: 특정 열(들)을 제거.
- `df.drop(index_number)`: 특정 행을 제거.
- 왜 제거하는가?
 - 분석에 필요 없는 열(e.g., ID),
 - 중복된 정보,
 - 너무 많은 고유값을 가져 모델 성능에 방해가 되는 열 등을 제거.

```
# deck 열 삭제하기
df.drop('deck', axis=1, inplace=True)
df.info()
```

inplace 옵션

- inplace=False (기본값)
 - 원본은 그대로 두고, 변경된 새로운 DataFrame을 반환
 - 변경된 결과를 사용하려면 새로운 변수에 할당하거나, 원래 변수에 덮어써야 함

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'A': [1, 2, np.nan], 'B': [4, np.nan, 6]})

# 원본 df는 바뀌지 않음
new_df = df.fillna(0)

print("--- 원본 df ---")
print(df)
print("\n--- new_df (결과) ---")
print(new_df)
```

--- 원본 df ---

	A	B
0	1.0	4.0
1	2.0	NaN
2	NaN	6.0

--- new_df (결과) ---

	A	B
0	1.0	4.0
1	2.0	0.0
2	0.0	6.0

inplace 옵션

- inplace=True
 - 원본 DataFrame을 직접 수정하고, 아무것도 반환하지 않음 (None)
 - 변수에 다시 할당할 필요가 없음

```
df = pd.DataFrame({'A': [1, 2, np.nan], 'B': [4, np.nan, 6]})
```

```
# df 자체가 수정됨, 반환값은 None  
result = df.fillna(0, inplace=True)
```

```
print("--- 원본 df (수정됨) ---")  
print(df)  
print(f"\n--- 반환값 ---")  
print(result)
```

--- 원본 df (수정됨) ---

	A	B
0	1.0	4.0
1	2.0	0.0
2	0.0	6.0

--- 반환값 ---

None

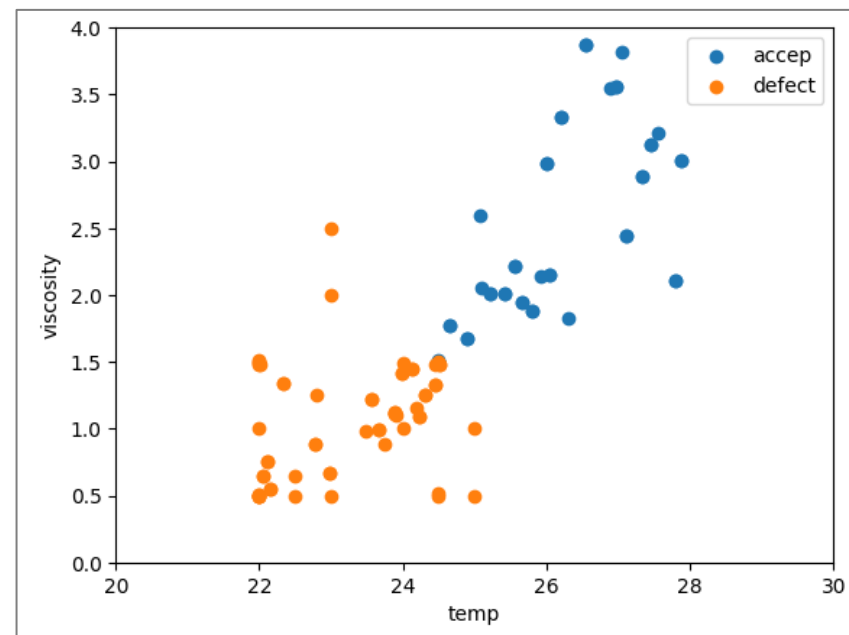
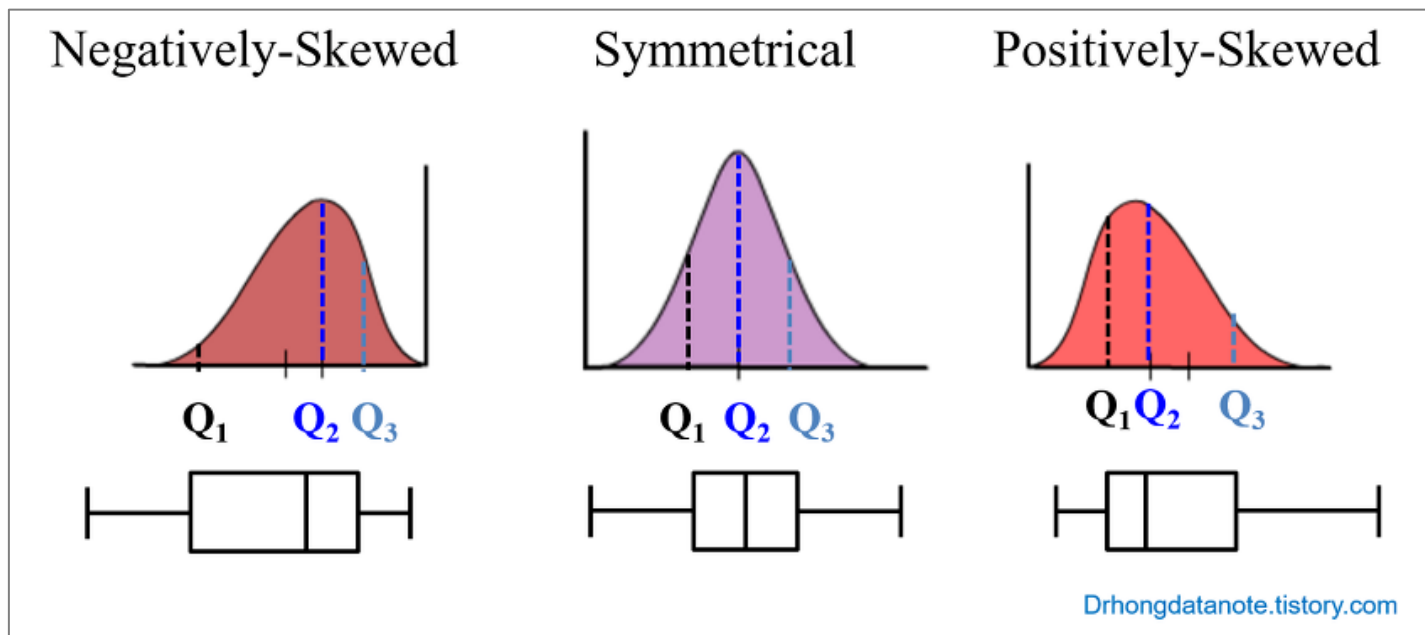
과거에는 inplace=True가 메모리를 절약한다는 장점 때문에 종종 사용되었지만, 최근에는 inplace=False (기본값) 사용이 더 권장

이상치 (Outliers)

- 정의: 다른 데이터 포인트들과 비교했을 때, 극단적으로 동떨어져 있는 값을 의미
 - 예를 들어, 학생들의 키를 측정하는데 250cm나 100cm 같은 값이 있다면 이상치로 볼 수 있음
- 이상치의 영향
 - 데이터의 평균이나 표준편차를 왜곡시킴
 - 선형 회귀(Linear Regression)나 SVM처럼 거리에 민감한 모델의 학습을 방해할 수 있음

이상치 (Outliers)

- 이상치 탐지 방법
 - 시각화 활용 : Box Plot, 산점도
 - 통계적 기법 활용 : Z-Score



이상치 처리 전략

① 삭제: 이상치로 판단된 행 전체를 제거

② 대체: 이상치를 다른 값으로 바꿈

- 평균(Mean) / 중앙값(Median) / 최빈값(Mode)으로 대체

- : 중앙값은 이상치의 영향을 덜 받기 때문에 일반적으로 평균보다 더 선호됨

- 상한/하한 설정 (Capping/Winsorizing)

- : 이상치를 특정 분위수(예: 1%, 99%) 값으로 설정합니다.

- : 예를 들어, 하위 1%보다 작은 값은 모두 1% 값으로, 상위 99%보다 큰 값은 모두 99% 값으로 강제로 맞춤

- : 데이터의 극단적인 분포를 부드럽게 만들어줍니다.

이상치 처리 전략

③ 변환 : 데이터 전체에 특정 수학 함수를 적용하여 이상치의 영향을 줄임

- 로그변환
- 제곱근 변환

④ 강건한 모델 사용 (Robust Model)

- 데이터를 직접 건드리는 대신 이상치에 덜 민감한 머신러닝 모델을 사용
 - : 트리 기반 모델이 대표적

이상치 처리 전략: 어떤 방법을 선택해야 할까?

- 이상치의 원인 파악이 최우선

- "가장 좋은 방법"은 정해져 있지 않으며, 데이터와 문제의 종류에 따라 달라짐

측정 오류나 데이터 입력 실수일 경우	• 삭제하거나 수정(대체)
이상치가 중요한 정보일 경우	• 사기 탐지, 불량품 예측 등에서는 이상치 자체가 가장 중요한 데이터 • 이 경우 삭제해서는 절대 안 됨 • 이상치 탐지 모델(Anomaly Detection)을 적용
데이터가 충분하지 않을 경우	• 삭제보다는 대체나 변환을 고려하는 것이 좋음
모델 선택의 자유도가 높을 경우	• 데이터 수정을 최소화하고 싶다면 강건한 모델을 사용 • (예: Random Forest)

중복 데이터 제거

```
# 중복 데이터 확인
df.duplicated().sum() # duplicated() 중복된 행에 True를 반환

#df.duplicated()
```

np.int64(1)

```
# 중복 데이터 출력
df[df.duplicated()]
```

```
#df.loc[df.duplicated(), :] # 불리언 인덱싱
```

	sepal_length	sepal_width	petal_length	petal_width	Target
142	5.8	2.7	5.1	1.9	2

```
# 중복 데이터 모두 출력
# df.loc[(df.sepal_length==5.8)&(df.petal_width==1.9), :]

df[(df.sepal_length==5.8)&(df.petal_width==1.9)]
```

	sepal_length	sepal_width	petal_length	petal_width	Target
101	5.8	2.7	5.1	1.9	2
142	5.8	2.7	5.1	1.9	2

```
# 중복 데이터 제거
df = df.drop_duplicates()

#df.loc[(df.sepal_length==5.8)&(df.petal_width==1.9), :]
df[(df.sepal_length==5.8)&(df.petal_width==1.9)]
```

	sepal_length	sepal_width	petal_length	petal_width	Target
101	5.8	2.7	5.1	1.9	2

범주형 데이터 인코딩

Pandas에서 문자열 다루기

- str 접근자
 - Series 객체 내 문자열 데이터에 파이썬 문자열 메서드를 적용할 수 있게 해주는 도구
 - : Series 전체는 하나의 문자열이 아니므로 series.lower()와 같이 사용할 수 없으나,
 - : series.str.lower()와 같이 사용하면 Series의 각 요소에 대해 문자열 함수를 적용할 수 있음

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Kim, Yuna', 'Park, Jisung', ' Son, Heungmin '],
    'City': ['Gyeonggi-do', 'Seoul', 'Gangwon-do']
})

# 모든 문자열을 소문자로 변경
df['Name'].str.lower()
```

	Name
0	kim, yuna
1	park, jisung
2	son, heungmin

dtype: object

데이터 인코딩

- 결측치를 모두 처리한 깨끗한 데이터를 준비했다. 그 다음에는?
- 머신러닝 모델은 문자 데이터를 인식하지 못함.
 - : 'sex' 열의 'male', 'female'이나 'embarked' 열의 'S', 'C', 'Q' 같은 문자열 데이터를 머신러닝 모델은 계산할 수 없다. 모델은 오직 숫자만 이해한다!
- 문자로 구성된 범주형 데이터를 숫자로 바꾸는 작업이 필요 == 데이터 인코딩

레이블 인코딩 label encoding

- 범주형 데이터를 숫자로 일대일 매핑해주는 인코딩 방식
 - ex) 과일 문자열 데이터의 값에 사전순으로 번호를 매겨 레이블 인코딩한 예시

원본	레이블 인코딩 적용 후
사과	3
블루베리	2
바나나	1
귤	0
블루베리	2
바나나	1
바나나	1
사과	3

레이블 인코딩 label encoding

- map() 함수
 - 특정 열(Series)의 각 값들을 다른 값으로 일대일 변환(mapping)할 때 사용.
 - 주로 딕셔너리로 값 변환
 - 키(key)는 기존 값, 값(value)은 바꿀 값으로 구성된 딕셔너리를 전달

```
# map() 예제
ex_s = pd.Series(['S', 'C', 'Q', 'S'])

# 일대일 변환할 데이터: 약어를 전체 이름으로 변경
embarked_map = {'S': 'Southampton', 'C': 'Cherbourg', 'Q': 'Queenstown'}

ex_s_mapped = ex_s.map(embarked_map)

print(ex_s)
print("-"*20)
print(ex_s_mapped)
```

```
0    S
1    C
2    Q
3    S
dtype: object
-----
0    Southampton
1    Cherbourg
2    Queenstown
3    Southampton
dtype: object
```

레이블 인코딩 label encoding

- apply() 함수
 - map()으로는 처리하기 어려운 복잡한 로직을 적용하고 싶을 때 사용.
 - if/elif/else와 같은 복잡한 조건 로직을 단일 열의 모든 값에 적용하고 싶을 때

```
# apply()
df_apply = pd.DataFrame({'age': [5, 15, 25, 40]})

# 나이를 기준으로 세대 그룹을 반환하는 함수
def get_age_group(age):
    if age < 13:
        return 'Child'
    elif 13 <= age < 20:
        return 'Teenager'
    else:
        return 'Adult'

# 'age' 열(Series)에만 apply 함수를 적용
df_apply['age_group'] = df_apply['age'].apply(get_age_group)

print(df_apply)
```

	age	age_group
0	5	Child
1	15	Teenager
2	25	Adult
3	40	Adult

레이블 인코딩 label encoding

- 예제:
 - Sex 열을 map 함수를 사용해 숫자(0, 1)로 변환하고
 - 'Numeric_Gender'라는 새로운 열에 저장하기

```
# 성별을 숫자형 데이터로 변환하기
```

```
df['Numeric_Gender'] = df['Sex'].map({'male': 0, 'female': 1})  
df['Numeric_Gender']
```

```
# 성별을 숫자형 데이터로 변환하기
```

```
df['Num_Gender_a'] = df['Sex'].apply(lambda x: 0 if x == 'male' else 1)  
df['Num_Gender_a']
```

Numeric_Gender	
0	0
1	1
2	1
3	1
4	0
...	...
886	0
887	1
888	1
889	0

레이블 인코딩의 함정

- 'embarked' 열(S, C, Q)을 0, 1, 2로 바꾸면 어떤 문제가 생길까?
 - 모델이 숫자 크기 자체를 학습에 반영할 수 있다.
 - 이를테면, Q(2)가 S(0)보다 2배 더 중요하다는 식의 잘못된 학습
 - 즉, 순서가 없는 명목형 데이터에 순서가 있는 것처럼 오해를 불러일으킨다.
- 머신러닝 모델은 서로 가까운 숫자를 비슷한 데이터라고 판단한다.
 - 과일 예제에서 1(바나나)과 3(사과)보다 1(바나나)과 2(블루베리)를 더 비슷한 데이터라고 인식할 수 있다. 그러나 바나나, 사과, 블루베리는 독립적 데이터.

-> 해결책: 원-핫 인코딩 (One-Hot Encoding)

원-핫 인코딩 (One-Hot Encoding)

- 여러 값 중 하나^{one}만 활성화^{hot}하는 인코딩
 - : 데이터에 포함된 고유값(Category)의 개수만큼 새로운 열을 만들고,
 - : 해당하는 열에만 1을 표시하고 나머지는 0으로 채우는 방식.
 - : 데이터의 '존재 유무'만 표현하므로 순서의 왜곡이 없다.

원본	원-핫 인코딩 적용 후			
과일	과일_귤	과일_바나나	과일_블루베리	과일_사과
사과	0	0	0	1
블루베리	0	0	1	0
바나나	0	1	0	0
귤	1	0	0	0
블루베리	0	0	1	0
바나나	0	1	0	0
바나나	0	1	0	0
사과	0	0	0	1

원-핫 인코딩 (One-Hot Encoding)

- `get_dummies()` 함수
 - 데이터프레임의 특정 열을 원-핫 인코딩된 새 열들로 간단하게 바꿔주는 함수
 - 동작 방식
 - : 변환할 열에서 고유한 값들을 모두 찾습니다. (예: 'sex' 열의 'male', 'female')
 - : 각 고유한 값을 새로운 열의 이름으로 만듭니다. (예: 'sex_male', 'sex_female')
 - : 원래 데이터의 값에 해당하는 새로운 열에 1을 표시하고, 나머지 열에는 0을 표시합니다
 - Pandas 2.0 버전부터는 기본 데이터 타입이 bool(True/False)로 나옴

```
df = pd.DataFrame({  
    'sex': ['male', 'female', 'male'],  
    'embarked': ['S', 'C', 'S']  
})  
  
pd.get_dummies(df)
```

	sex_female	sex_male	embarked_C	embarked_S
0	False	True	False	True
1	True	False	True	False
2	False	True	False	True

원-핫 인코딩 (One-Hot Encoding)

- `get_dummies()` 주요 옵션
 - `prefix`:
 - 생성될 가변수 열 이름에 접두사를 붙여 어떤 원래 열에서 왔는지 명확하게 함
 - `drop_first=True`:
 - 각 범주의 첫 번째 값을 제거하여 열의 개수를 하나 줄임 (다중공선성 문제를 방지하기 위해 사용)
 - `sex_male`이 0이면 당연히 `sex_female`은 1이므로 두 열 중 하나는 사실상 불필요한 정보

```
# 주요 옵션 : prefix
pd.get_dummies(df_dum_ex, prefix=['gender', 'port'])
```

	gender_female	gender_male	port_C	port_S
0	False	True	False	True
1	True	False	True	False
2	False	True	False	True

```
# 주요 옵션 : drop_first
pd.get_dummies(df_dum_ex, drop_first=True)
```

	sex_male	embarked_S
0	True	True
1	False	False
2	True	True

원-핫 인코딩 (One-Hot Encoding)

- 다중공선성 (Multicollinearity)
 - 통계학, 특히 회귀분석에서 독립 변수들 간에 강한 상관관계가 나타나는 현상
 - 회귀 계수 추정의 정확성을 떨어뜨리고 모형의 해석 및 예측을 어렵게 만드는 문제점
 - 다중공선성은 독립 변수들 사이에 강한 선형 관계가 존재할 때 발생
 - : 즉, 하나의 독립 변수를 다른 독립 변수들의 조합으로 표현할 수 있는 경우를 의미
 - 원-핫 인코딩된 열들은 다중공선성 문제를 일으킬 수 있음.
 - : 하나를 알면 나머지를 유추할 수 있으므로
 - ex) embarked_S=0, embarked_C=0 이면 embarked_Q는 반드시 1
 - : 열 하나를 제거해도 정보 손실이 없고 다중공선성 문제를 방지할 수 있음.
 - drop_first=True 옵션 사용

끝!